



Use Databases to Develop Apps on QT

This article covers how to use Qt with databases—the backbone of many applications.

The Qt framework provides a set of APIs (classes) that are used to access databases. Qt supports MySQL, PostgreSQL, SQLite, Sybase, Oracle, IBM DB2, and Borland Interbase. We will use MySQL for the examples in this article.

If you haven't already installed MySQL server, do so. I used CentOS 6.0 while writing this article; its installation DVD carries MySQL, making it a quick local install using *yum install mysql*.

Use the method you prefer on your distro to install MySQL. And consult your distro's documentation for service management (starting and stopping the service, etc). It also makes sense to learn the basics of SQL if you are not well versed with it, because I will not cover that here.

Database connection

Create a Qt project, as explained in previous articles. The new thing to do is add the SQL module in the project (.pro) file, as follows:

```
QT += core gui sql
```

This will add the Qt SQL classes to your application. To understand how to connect with the database, let's right away examine some sample code:

```
#include <QtGui/QApplication>
#include "dialog.h"
#include <QtGui>
#include <QtCore>
```

```
#include <QtSql>
int main(int argc, char *argv[]) {
    QApplication a(argc, argv);
    Dialog w;
    w.show();

    QSqlDatabase db = QSqlDatabase::addDatabase("QMYSQL");
    db.setHostName("localhost");
    db.setDatabaseName("osslabs");
    db.setUserName("oss");
    db.setPassword("oss123");

    if (!db.open()) {
        qDebug() << db.lastError();
    }
    qDebug() << "Database opened successfully";
    return a.exec();
}
```

The class that deals with the database connection is *QsqlDatabase*; you need to provide a database driver name to the constructor — 'QMYSQL', in our case. Next, set the host name where the MySQL server is running. In this case, it's *localhost*. Then set the database name, username and password. Now open the database. If it's not successfully opened, you can get the error from the *lastError()* member function.

Executing queries

The next task is to execute some queries. For this, Qt provides the *QSqlQuery* class. Here's the code: